

HLiquity Emergency Exit: Atomic Batch- Transaction Implementation Guide — Community Edition

How anyone maintaining or mirroring a HLiquity frontend can close the allowance-sweep race window for every user exit flow, without touching the immutable mainnet contracts: HIP-551 atomic batch transactions now, a permissionless redemption router in parallel, and a disciplined wind-down ahead of Hedera's March 2027 removal of smart-contract calls from atomic batches. Includes a complete no-frontend CLI walkthrough and the frontend build, signing-model, UX, security, and test-plan specification. Everything in this document is a proposal that community contributors can adopt, adapt, or take over — nobody mandates anything. **Mainnet-validated (community-2.3):** a community contributor reproduced the batched redemption on mainnet — see §3.3 — and the CLI now uses the same JSON-RPC simulation path after Hedera rejected an SDK local query for `fetchPrice()`.

Date: 2026-10-05 **Version:** community-2.3 **Classification:** Public — community security documentation

Revision: community-2.3: CLI redemption preflight moved to frontend-equivalent JSON-RPC `eth_call`; raw ED25519/ECDSA key parsing corrected; both mainnet observations recorded — changelog in the appendix (§9.4).

Published by the HLiquity community. No entity operates the HLiquity protocol; this document was produced by community contributors from public mainnet data. Not financial advice. Corrections via GitHub issues only.

01 TL;DR — executive summary

- **The race window is the vulnerability users can close.** The 2026-09-29 drain executed ~7 seconds after the victim's approval: between the approve transaction and the redeem transaction, the HCHF allowance stood live on-chain and the sweeper bot spent it.
CONFIRMED (community forensic analysis of mainnet data — methodology in §2 and the appendix)
- **No on-chain fix exists.** The HLiquity core contracts are immutable and ownership-renounced — verified on-chain. Every mitigation is off-chain; this document is the engineering guide for that mitigation. **CONFIRMED**
- **The mechanism: HIP-551 atomic batches.** Put `[CryptoApproveAllowance → ContractCall]` into one batch: either every inner transaction commits or all roll back. Mainnet has supported Atomic Batch Transactions since 2025-10-09 (services v0.66). **CONFIRMED**
- **Our shape fits the network rule exactly:** since 2026-09-29 (services v0.77) a batch may contain at most ONE contract operation, and it must be the last inner transaction. Approve-first + redeem-last complies; two contract calls in one batch is not possible. **CONFIRMED**
- **Proven on mainnet by a community contributor (§3.3).** The batched redemption was reproduced on mainnet: inner 1 native `CryptoApproveAllowance` (1 HCHF), inner 2 `TroveManager.redeemCollateral`, one parent AtomicBatch — succeeded, exactly 100,000,000 units consumed and burned, HBAR returned, zero residue allowance; mirror record confirms ordering and shared parent. **Three signing prompts observed.** The assembled wallet flow is upgraded from UNVERIFIED to CONFIRMED (community-validated).
CONFIRMED
- **Batching removes the inter-transaction gap** in which the bot operates: allowance is set and consumed inside a single consensus transaction, with zero residue if the batch rolls back. This is the only change that closes the 7-second window without touching a contract.
RECOMMENDATION **INFERENCE** (mechanism CONFIRMED; residual-risk claim §04/§08)
- **Hard sunset.** Hedera deprecated smart-contract calls as batch inners on 2026-09-22, with removal planned for “March 2027, estimated”. Batches are a ~5-month bridge, not the endgame. **CONFIRMED** (VERIFIED-NARROWED, §03)
- **Native batches survive the sunset — but that does not help exits.** `CryptoApproveAllowance` + `CryptoTransfer`-style batches remain supported; every HLiquity user exit still needs a contract call, so batches alone cannot carry users past March 2027. **CONFIRMED**
- **Durable fix: a permissionless redemption router** that composes approve+redeem inside one EVM call (user approves the router; the router self-grants wrapper allowance only inside the call and zeroes it after). Survives the sunset. Scope is **redemption only**: `closeTrove/adjustTrove` act on `msg.sender`, so a router cannot perform them for a trove owner — existing-user trove operations ride batches until the cutoff, migration, or v2.

StabilityPool exits, unstaking, and claims need no router at all (no allowance; standalone calls keep working after March 2027). **RECOMMENDATION**

- **Endgame: v2** with fixed contracts and a trustless merkle-snapshot migration — the essentials of the community migration analysis are summarized in §3.2. Independent of this document. **RECOMMENDATION**
- **Deadline math:** 2026-10-05 → March 2027 ≈ 20–22 weeks. All batched user exits complete by **2027-01-31**; nothing may depend on contract-call inners past February 2027.
INFERENCE (from the “estimated” caveat, §03)
- **Frontend path is cheap.** The blokk fork already runs `@hashgraph/hedera-wallet-connect` ^1.4.2 + `@hashgraph/sdk 2.72.0` (≥2.64 required) and builds native `ContractExecuteTransactions` — the batch migration is a wrapper change, not a stack change. **Do not touch hashconnect** (deprecated, repo gone). Minimum wallet: HashPack ≥14.4.0. **CONFIRMED**
- **Signing pattern validated on mainnet (§3.3, §6.4):** generate a one-use ED25519 batch key; sign each frozen inner separately in HashPack and retain each transaction returned by `DAppSigner.signTransaction`; assemble and wallet-sign the outer; add the ephemeral batch-key signature locally; relay the already fully signed bytes with `hedera_executeTransaction`. Three HashPack prompts were observed. The untested single-prompt path is not offered. **CONFIRMED** (community-validated)

Open community decisions — nothing here is mandated; each item is a proposal contributors can adopt, adapt, or take over:

1. **Batch-every-allowance-flow** (7 flows, §06 map) with exact-amount approvals and residue verification (immediate allowance check after every committed batch) — *proposed approach*.
2. **Router contract** — permissionless-redemption scope per the narrowed §3.2 — built in parallel, go/no-go by week 8 (§07) — *looking for a contributor to lead this track*.
3. **Wallet floor:** HashPack ≥14.4.0 as the *recommended minimum*; block older, unknown, and other wallets. There is no safe non-atomic fallback.
4. **Timeline:** *proposed* smallest-amount mainnet validation → controlled rollout per §07, including the 2027-01-31 completion buffer. The published registry is mainnet-only.
5. **V2:** *independent track* per the community migration analysis — it must not block the router.

02 Incident recap — what happened and what it cost

All incident facts below come from community forensic analysis of public mainnet data (methodology in this section and the appendix); this section carries only what the build decision depends on. **CONFIRMED**

Root cause. CWE-862 (Missing Authorization) confused deputy: the HLIquity token wrappers inherit an ungated `transferFrom` helper (selector `0x15dacbea`) that spends any standing HTS allowance without checking who is asking. Users grant allowances to the *wrapper* contracts (all HTS pulls execute via precompile `0x167`), which is the vulnerable surface. The attacker's sweeper bot — gas relayer 0.0.995584, receiver 0.0.10393273 — watched for new allowance grants and spent them, typically within seconds. Reference transaction [0.0.995584-1790680920-326001014](#) (view on HashScan): 1,800 HCHF drained from victim 0.0.846778, **7 seconds after the victim's approval**. The victim's subsequent protocol redeem executed and reverted atomically (HTS status 292, `SPENDER_DOES_NOT_HAVE_ALLOWANCE`) — the protocol behaved correctly; the loss happened in the gap between the two user transactions.

2.1 The three critical approvals (the race window lives here)

#	TOKEN	USER MUST APPROVE (SPENDER)	CONSUMED BY
1	HCHF 0.0.6070123	HCHFToken wrapper 0.0.6070122	provideToSP, repayHCHF, closeTrove, adjustTrove (repay side), redeemCollateral
2	HLQT 0.0.6070128	HLQTTToken wrapper 0.0.6070127	HLQTStaking.stake
3	LP 0.0.6070469	SaucerSwapPool 0.0.6070133	SaucerSwapPool.stake

Every one of these flows becomes safe when the approval and the consuming call execute atomically — that is the entire design target of this guide. Contract set verified on-chain 2026-10-05 by community fingerprinting (full set in §05 step 3); repo-committed configs are stale and must not be used (§05 step 3, risk §08).

closeTrove approves CURRENT NET DEBT, never typed/displayed debt (safety-critical).

`BorrowerOperations.closeTrove` applies pending rewards, then burns **current entire debt – 20 HCHF** from the user and takes the 20 HCHF gas compensation from the GasPool. Read `getEntireDebtAndColl(user)` immediately before building; a user-entered overestimate leaves a sweepable allowance. **CONFIRMED** (repo-verified)

2.2 Campaign and dollar context

CAMPAIGN	HLIQUITY LEG	REFERENCE
4 waves, 2026-07-08 → 2026-10-03; 10 victims, 5,907.60 HCHF	Jul 8: seven-victim burst in 77 s · Sep 4: 1,750.41 HCHF · Sep 29: 1,800 HCHF · Oct 3: 1,780.66 HCHF	Community forensics (mainnet data) CONFIRMED
Full-campaign forensics (all venues)	165 relay calls → 119 committed → 113 distinct debited accounts; plus BankSocial leg: 98 transfers (12.23M BSL + 101.97M sBSL + 6 rUSD via deleted contract 0.0.7444023)	Community forensics (mainnet data) CONFIRMED
Current open- allowance exposure	\$22.18 across 19 owners — small but recurring with every new grant	Community forensics (mainnet data) CONFIRMED

MEASURE (US\$)	VALUE	BASIS
HLiquity observed damage	\$7,451.06 (HCHF portion ≈ \$7,113)	Community valuation at historical on-chain marks; HCHF ≈ \$1.204 (CoinGecko 2026-10-04)
Same-class losses since 2023	≈ \$611K–\$636K	Includes 2023 precompile attack ~\$600K + 2026 campaign venues (community campaign ledger)
BankSocial sweep (same crew)	\$3,036 – \$28.4K (pricing range)	BSL priced; upper bound marks illiquid sBSL at parity
WHBAR, measured 16-day window	≈ \$790 (7,725.34 WHBAR)	Mirror-node sample, \$0.1022 (CoinGecko 2026-10-04)
WHBAR wrapper exposure (not a loss)	\$20.35M supply (199.1M WHBAR, 1,003+ holders)	The live surface that dwarfs all realized losses (community wrapper-allowance audit)
Hedera-wide exploit \$ since 2023	≈ \$11.6M+ (Sirio ~\$2M Feb 2025; Bonzo ~\$9M Jul 2026)	Bonzo attribution caveat: oracle per Bonzo, LayerZero per third parties
Comparison anchor	HLiquity is small in \$ next to Bonzo (\$9M) and Sirio (\$2M), but identical in mechanism to the 2023 ~\$600K event and part of the only recurring exploit family on Hedera (~5% of dollars, 2-of-4 major incidents, the only recurring class). CONFIRMED	

03 The external expert assessment, verified

An external engineering assessment of the incident reached community contributors with three candidate mitigations and a warning that the obvious one might be short-lived. Community contributors verified every load-bearing claim against primary sources (HIP, Hedera docs and consensus-node source, the deprecation blog, npm, wallet patch notes, both GitHub forks).

Verdict: **the assessment is correct on all three options and on the deprecation, with one narrowing** — the March 2027 date is explicitly an estimate. **CONFIRMED**

“Option 1 — batch transactions: put the approval and the protocol call into a single HIP-551 atomic batch so there is no exposed window. Option 2 — redemption router: a contract that composes approve-and-operate inside one EVM call; durable, but on its own it leaves trove operations behind. Option 3 — v2 redeploy: fixed contracts with a trustless migration, per the community’s migration analysis.”

External engineering assessment received by the community (paraphrase; German original available in the issue tracker). Option numbering is this document's. The assessor's "router only redeems" concern stands as a hard scope constraint — a router cannot carry closeTrove/adjustTrove for existing users (they act on `msg.sender`): see the narrowed scope in §3.2 and §08.

3.1 Fact-check: the March 2027 deprecation VERIFIED-NARROWED

Fact-check box — the deprecation the assessment warned about

Source: official Hedera blog, 2026-09-22 — "Atomic Batch Transactions No Longer Support Smart Contract Calls." CONFIRMED (the blog exists and says exactly this) · INFERENCE (treating "estimated" as a hard planning date).

"Smart contract calls submitted as inner transactions of an Atomic Batch Transaction enter a six-month deprecation period, with removal planned for March 2027. Atomic batch transactions remain fully supported for composing native Hedera service operations." Blog quote 1 — deprecation + what survives "As of September 2026: a batch may contain no more than one smart contract call, and it must be the final inner transaction." Blog quote 2 — current constraint (matches services v0.77 behavior) "From March 2027: smart contract calls are no longer supported as inner transactions of an atomic batch." Blog quote 3 — the removal "March 2027, estimated." Blog, timeline — the narrowing: no committed date

Load-bearing implication. Our exact mitigation — approve + ContractCall in a batch — **stops working around March 2027**. No official March-2027 ban on native batches, atomic swap, or allowance behavior exists (searched; none). Stated rationale for the removal: EVM compatibility — "no equivalent construct in the EVM." Precedent says Hedera executes such deprecations: the 2023-03-09 precompile attack (~\$600K) was answered first by an emergency proxy-access shutoff and then permanently in services v0.34.5 (blocking delegatecall→precompile), followed by the HSCS Security Model v2. Plan for the sunset as real, keep the buffer, and do not let the router depend on batch inners at all.

3.2 What this means for the three options

OPTION	VERDICT	ROLE
1 — Atomic batch transactions	Verified against HIP-551, services source, and SDK — and now reproduced on mainnet by a community contributor (§3.3): works today and closes the window immediately. CONFIRMED	Deploy now (weeks) — the stop-gap bridge until ~March 2027
2 — Redemption router	Sound and durable past the sunset (composition inside the EVM is Hedera's own recommended replacement). Scope narrowed (community-2.0): a durable router can carry permissionless redemption only — closeTrove/adjustTrove act on <code>msg.sender</code> , so a router would be the caller, not the trove owner, and cannot perform those operations for existing users. SP exits, unstaking, and claims need no router (no allowance involved; standalone calls keep working after March 2027). Existing-user trove operations = batches until cutoff / migration / v2. RECOMMENDATION	Build in parallel (weeks 3-16) — the durable redemption path
3 — v2 redeploy	Correct endgame; a separate community migration analysis exists, and the essentials are summarized here: immutable contracts → all mitigations off-chain → v2 with trustless merkle-snapshot migration is the endgame. RECOMMENDATION	Decision track, weeks 5-20 — independent of this guide

Why “native batches stay supported” matters — and why it does not save the exits. After the sunset, CryptoApproveAllowance + CryptoTransfer-style atomic batches remain fully supported. But no allowance-carrying HLiquidity exit can be expressed without a contract call: redeeming HCHF, closing a trove, repaying, adjusting, staking — all are `ContractCall`s. The surviving native capability is therefore irrelevant to those exits, and the batch is correctly sized as a bridge, with the router as the durable crossing *for redemption*. Operations with no allowance — StabilityPool withdrawals, unstaking, claims, `claimCollateral` — keep working as ordinary standalone contract calls after March 2027 (no batch, no router needed). The allowance-carrying trove operations must complete inside the batch window — or, for redemption, move to the router. **CONFIRMED**

3.3 Community mainnet validation — PROVEN

Community mainnet validation — proven **PROVEN**

A community contributor reproduced the batched redemption **end-to-end on mainnet** and reported it in the GitHub issue (contributor unnamed unless they request attribution; test evidence in the issue). **CONFIRMED** (community-validated)

- **Reproduction:** inner 1 = native `CryptoApproveAllowance` (1 HCHF = 100,000,000 units, 8-dec) to the HCHF wrapper; inner 2 = `TroveManager.redeemCollateral` as the final inner — both under one parent `AtomicBatch`. Exactly the shape this guide specifies, satisfying the one-contract-op-last rule in practice.
- **Result:** the batch succeeded; **exactly 100,000,000 units consumed and burned**; HBAR collateral returned to the user; **residue allowance zero**.
- **Mirror record** confirms the inner ordering and the shared parent (type `ATOMICBATCH`; inners linked by `parent_consensus_timestamp` / `nonce`).
- **Signing observed: three signing prompts** — inner 1, inner 2, outer batch — i.e. per-transaction signing via hedera-wallet-connect 1.5.1, with the signed bytes relayed via `hedera_executeTransaction` (details in §6.4 and the issue).

Status upgrades (UNVERIFIED → CONFIRMED, community-validated): the exact ephemeral-batch-key flow described in §6.2; retaining every signer-returned inner and outer; HashPack ≥14.4.0 batch signing via connector 1.5.1.

Not established by this test: a one-call signing API, other wallets/connectors, other protocol flows, or how HashPack renders each prompt.

3.4 Community guidance — what members should tell each other

This section is written to be quoted directly — in Discord, Telegram, X, or a README banner. Short version first, then the details behind it.

Member fact-sheet — quote freelyShareable

- **What happened.** A sweeper bot has been draining standing token allowances on Hedera — including HLiquidity's HCHF — typically within seconds of a user's approval. The HLiquidity leg: 10 victims, 5,907.60 HCHF, four waves from 2026-07-08 to 2026-10-03. The protocol contracts behaved correctly; the loss happens in the seconds between a user's approve and the user's redeem. **CONFIRMED**
- **Who is at risk.** Anyone holding an open (unspent) HCHF, HLQT, or LP allowance to the wrapper contracts — the allowance stays sweepable until it is spent or revoked — and anyone who creates a fresh non-atomic approval.
- **What to do now.** (1) Do not create fresh non-atomic HCHF/HLQT/LP approvals. (2) Revoke existing allowances. (3) For redemption, use the exact mainnet-tested browser signing flow and chunks no larger than 1,780 HCHF; reject truncation. Other flows remain source-reviewed, not mainnet-proven. For closeTrove, read current entire debt and approve exactly debt – 20 HCHF. (4) Verify every address against the manifest. **RECOMMENDATION**
- **What never to do.** Never approve “unlimited” amounts. And: nobody will ever DM you a refund — anyone offering compensation is a scammer. Always. **CONFIRMED** (as community policy)

Scam warning — expect refund-impersonation waves. Publishing this document will likely trigger DMs from accounts impersonating “the HLIquity team”, a “HLiquity foundation”, or a “compensation desk”. The community never DMs first, has no treasury, and sends no compensation. There is no refund flow, no compensation program, and no official support account. Treat every unsolicited refund offer as a scam; report and block, and warn others using the fact-sheet above.

How to contribute

- **Review this document** — corrections, however small, via GitHub issues.
- **Run it** — execute the §6.7 regression plan against the exact mainnet-tested signing code with the smallest practical amounts; the bundled registry is mainnet-only.
- **Pick up a track** — the frontend work (§06) and the redemption-router track (§3.2, §08) both need owners.
- **Improve the IOC list** — new sweeper addresses, watcher rules, flagging evidence.

No permission needed — that’s the point.

How to verify

Every address, transaction, and claim in this document is checkable against public infrastructure. Two endpoints cover almost everything:

- **Mirror node REST:** `https://mainnet-public.mirrornode.hedera.com/api/v1` — transactions, allowances, token relationships (§9.2 query list).
- **HashScan:** `https://hashscan.io/mainnet` — the reference drain transaction is at `/transaction/0.0.995584-1790680920-326001014`.

Don’t trust — verify, then open an issue if something doesn’t match.

04 Why atomic batches close the race

The sweeper bot wins because a live allowance exists between two transactions. HIP-551 closes that inter-transaction gap, but atomicity alone does **not** guarantee zero residue when a successful contract call consumes less than approved. For redemption, restrict each batch to one iteration, reject truncation, and keep the amount at or below `MIN_NET_DEBT = 1,780 HCHF`; then the first trove consumes the full request or the call reverts and the approval rolls back. Larger exits are repeated chunks. **CONFIRMED** (contract logic and 1 HCHF mainnet result)

TODAY — two separate transactionsexposed by construction
tx 1user signs `approve` — HCHF → wrapper 0.0.6070122, exact amount
approval confirms — allowance is now live and sweepable

EXPOSED WINDOW (7 s in the reference case). Sweeper bot 0.0.995584 spends the allowance via the wrapper's ungated `transferFrom ( 0x15dacbea )` — a plain HTS transfer, not a protocol interaction.

tx 2user's `redeemCollateral` executes — HCHF already gone; redeem reverts atomically (status 292). Loss already settled in tx 1's gap.

→ to retry, the user must re-approve — recreating the exact state the bot is watching for.

BATCHED — one atomic unit (HIP-551)no window exists — proven on mainnet (§3.3)

inner 1 `CryptoApproveAllowance` — HCHF → wrapper, exact amount (native op, any position)

inner 2 `ContractCall` — `TroveManager.redeemCollateral` (the ONE contract op; last by rule)

executes in listed order within a single consensus transaction — no inter-transaction gap

both commit — or both roll back (`INNER_TRANSACTION_FAILED`); redemption is accepted only in an exact-or-revert chunk ($\leq 1,780$ HCHF, one iteration, no truncation), then exact debit and zero residue are verified.

4.1 The one-contract-op-last rule — and why our shape fits

Mainnet constraint since 2026-09-29 (services v0.77, `AtomicBatchHandler.pureChecks`): a batch may contain at most ONE inner transaction that is a contract operation (`CONTRACT_CALL` , `CONTRACT_CREATE_INSTANCE` , or `ETHEREUMTRANSACTION`), and it must be the last inner; anything else pre-checks `INVALID_TRANSACTION_BODY` . Forbidden inner types (network blacklist): Freeze, AtomicBatch (no nesting), ScheduleCreate, ScheduleSign. Allowed native types include `CRYPTOAPPROVEALLOWANCE` and `CRYPTOTRANSFER` . Our mitigation shape `[CryptoApproveAllowance, ContractCall]` satisfies the rule exactly — confirmed in practice by the community mainnet reproduction (§3.3). Two contract calls in one batch (e.g. approve-token + redeem as two EVM calls) is **not possible** — approve must be the native HTS allowance transaction. **CONFIRMED**

4.2 Batch limits and cost model CONFIRMED

PROPERTY	VALUE	ENGINEERING CONSEQUENCE
Inner count	≤ 50 (network config; SDK JSDoc stale-says 25)	Plan ≤ 25 to be safe; our flows use 2 (3 with association)
Total size	≤ 6 KB per batch	7-arg redeemCollateral + approve fits comfortably
Valid duration	standard ~ 180 s	User must sign promptly; stale batches expire unspent
Inner node account	forced to 0.0.0; each inner has own payer + transactionId; dedup individually	Set explicit inner transactionIds when the inner payer differs from the client operator; re-submitting needs fresh ids
Fees	outer payer pays batch node+network fee; each inner pays its own full fees even if the batch later rolls back (inners before the failing one are charged)	A failed redeem costs approve-side fees too — show honest max-fee estimates; budget $2\times$ inner fees + batch fee
Signing	every inner needs a <code>batchKey</code> ; the outer must carry every distinct batch-key signature plus the payer signature	The tested browser path uses one fresh ephemeral ED25519 batch key on both inners, wallet-signs each inner and the outer in three prompts, then adds only the ephemeral signature locally

05 Tutorial: batch transactions without a frontend (CLI)

Every flow in this guide can be executed from a Node.js CLI — no frontend, no wallet extension, using only SDK APIs verified in §01/§03. A complete reference implementation of this tutorial (plus the §06 frontend pattern in TypeScript) is published alongside this document in the repository under `code/` (`code/cli` , `code/frontend`). The bundled registry is **mainnet-only**; rehearse transaction construction offline and begin with the smallest practical mainnet amount. RECOMMENDATION

Step 1 — Install

Node 18+. The BatchTransaction API requires `@hashgraph/sdk` $\geq 2.64.0$ (also shipped as `@hiero-ledger/sdk` 2.89.x). CONFIRMED

```
# package.json (excerpt)
{
  "type": "commonjs",
  "dependencies": {
    "@hashgraph/sdk": "^2.72.0",
    "dotenv": "^16.0.0",
    "ethers": "5.7.2"
  }
}
# install
npm install
```

Step 2 — Environment and key handling

The CLI runs with a payer account — in SDK terms the client *operator* — that pays every fee: the batch fee and each inner's full fee, including inner fees already spent when a batch later rolls back (§4.2). Keys come from the environment or a permission-locked file; never hardcode them, never commit them, never put them anywhere near frontend code. **CONFIRMED** (fee model) / **RECOMMENDATION** (handling)

```
# .env - chmod 600 .env; never commit
# MAINNET-ONLY reference: the bundled verified IDs do not exist on testnet.
# Rehearse construction offline and start with the smallest practical amount.
OPERATOR_ID=0.0.YOURACCOUNT
OPERATOR_KEY_DER=<DER-encoded key, or raw HashPack private key>
OPERATOR_KEY_TYPE=ed25519 # raw keys only; use ecdsa for an ECDSA wallet
JSON_RPC_RELAY_URL=https://mainnet.hashio.io/api

require("dotenv").config();
const { Client, PrivateKey, AccountId } = require("@hashgraph/sdk");

const keyText = process.env.OPERATOR_KEY_DER;
const keyType = (process.env.OPERATOR_KEY_TYPE || "ed25519").toLowerCase();
const operatorKey = PrivateKey.isDerKey(keyText)
  ? PrivateKey.fromStringDer(keyText)
  : keyType === "ecdsa"
    ? PrivateKey.fromStringECDSA(keyText)
    : PrivateKey.fromStringED25519(keyText);
const client = Client.forMainnet() // verified registry below is mainnet-only
  .setOperator(AccountId.fromString(process.env.OPERATOR_ID), operatorKey);
```

Step 3 — Pin the verified contract set (do not trust repo configs)

The mainnet deployment was never committed to the repo; the committed `hedera.json` is testnet-era and mislabeled. The set below was fingerprinted on mainnet on 2026-10-05.

CONFIRMED

NAME	CONTRACT ID	ROLE IN THE BATCHED FLOWS
BorrowerOperations	0.0.6070094	openTrove, addColl, withdrawColl, withdrawHCHF, repayHCHF, closeTrove, adjustTrove, claimCollateral
TroveManager	0.0.6070118	redeemCollateral (batched); liquidations (no batch)
StabilityPool	0.0.6070108	provideToSP (batched); withdrawFromSP / withdrawETHGainToTrove (no batch)
HLQTStaking	0.0.6070064	stake (batched); unstake (no batch)
SaucerSwapPool (LP rewards)	0.0.6070133	LP stake (batched); withdraw / claimReward / withdrawAndClaim (no batch)
HCHF wrapper / token	0.0.6070122 / 0.0.6070123	allowance spender for every HCHF-consuming flow
HLQT wrapper / token	0.0.6070127 / 0.0.6070128	allowance spender for HLQT staking
LP token (uniToken)	0.0.6070469	allowance token for LP staking
HintHelpers / PriceFeed	0.0.6070047 / 0.0.6070082	redeem hint derivation + price for the CLI pre-flight

Step 4 — Pre-flight: association check, price, and redemption hints

`redeemCollateral` needs a price and three hints. Obtain them with JSON-RPC `eth_call` simulations immediately before building the batch. Do **not** query `fetchPrice()` through an SDK `ContractCallQuery`: Hedera rejects that local query with `LOCAL_CALL_MODIFICATION_EXCEPTION` because the EVM path may update `lastGoodPrice`. JSON-RPC `eth_call` runs the execution-equivalent path while discarding state changes, matching the established frontend. When `partialUNICR` is non-zero, reproduce the frontend's `getApproxHint` → `findInsertPosition` sequence and pass the resulting upper/lower hints. The two-inner reference requires an already-associated token account (necessarily true when the account already holds HCHF).

CONFIRMED

```
const { ethers } = require("ethers");
const { randomBytes } = require("node:crypto");

const evm = id => "0x" + BigInt(id.split(".")[2]).toString(16).padStart(40, "0");
const relay = process.env.JSON_RPC_RELAY_URL || "https://mainnet.hashio.io/api";
const provider = new ethers.providers.StaticJsonRpcProvider(relay);
const priceFeed = new ethers.Contract(evm("0.0.6070082"),
  ["function fetchPrice() returns (uint256)"], provider);
const hintHelpers = new ethers.Contract(evm("0.0.6070047"), [
```

```

    "function getRedemptionHints(uint256,uint256,uint256) view returns
(address,uint256,uint256)",
    "function getApproxHint(uint256,uint256,uint256) view returns
(address,uint256,uint256)"
], provider);
const troveManager = new ethers.Contract(evm("0.0.6070118"),
["function getTroveOwnersCount() view returns (uint256)"], provider);
const sortedTrove = new ethers.Contract(evm("0.0.6070089"),
["function findInsertPosition(uint256,address,address) view returns
(address,address)"], provider);
const ZERO_ADDR = "0x000000000000000000000000000000000000000000000000";

// 4a) Verify HCHF association separately through mirror-node REST.
// 4b) Execution-equivalent simulation: redeemCollateral also calls fetchPrice().
const executionPrice = await priceFeed.callStatic.fetchPrice();

// 4c) HintHelpers.getRedemptionHints(amount, price, maxIterations) – maxIterations is a
// deliberate cap: higher scans more troves. `amount` is 8-decimal raw HCHF units.
const hints = await hintHelpers.callStatic.getRedemptionHints(
    amount.toString(), executionPrice.toString(), maxIterations);
const firstHint = hints[0];
const partialNICR = hints[1];
const truncatedAmount = hints[2]; // what the call can ACTUALLY consume

// 4d) For a partial redemption, derive live insertion hints exactly as the frontend
does.
let partialUpperHint = ZERO_ADDR, partialLowerHint = ZERO_ADDR;
if (BigInt(partialNICR.toString()) !== 0n) {
    const countResult = await troveManager.callStatic.getTroveOwnersCount();
    const troveCount = Number(countResult.toString());
    let remainingTrials = Math.ceil(10 * Math.sqrt(troveCount));
    let seed = BigInt("0x" + randomBytes(32).toString("hex"));
    let bestHint = ZERO_ADDR, bestDiff;

    // Cap each getApproxHint query at 2,500 trials, retaining the closest result.
    while (remainingTrials > 0) {
        const trials = Math.min(remainingTrials, 2500);
        const approx = await hintHelpers.callStatic.getApproxHint(
            partialNICR.toString(), trials, seed.toString());
        const diff = BigInt(approx[1].toString());
        if (bestDiff === undefined || diff < bestDiff) {
            bestDiff = diff;
            bestHint = approx[0];
        }
        seed = BigInt(approx[2].toString());
        remainingTrials -= trials;
    }

    const position = await sortedTrove.callStatic.findInsertPosition(
        partialNICR.toString(), bestHint, bestHint);
    partialUpperHint = position[0];
    partialLowerHint = position[1];
}

```

```
// 4e) EXACT-OR-REVERT RULE:
// require amount > 0, amount <= MIN_NET_DEBT (1,780 HCHF), maxIterations = 1,
// and truncatedAmount === amount. Otherwise abort. The first trove then either
// consumes the full request or redeemCollateral reverts before the approval commits.
// Split larger exits into repeated batches.
```

Two different mainnet observations. Parent batch `0.0.4093645@1791227892.358580307` returned `INNER_TRANSACTION_FAILED` ; its approval inner was `REVERTED_SUCCESS` , HCHF balance stayed unchanged, and the final allowance was zero. That validates atomic rollback, not redemption liveness. A later CLI run stopped before submission at transaction identifier `0.0.4093645@1791230284.744075766` with precheck `LOCAL_CALL_MODIFICATION_EXCEPTION` while SDK `ContractCallQuery` attempted `fetchPrice()` ; therefore no batch or allowance was created in that run. The reference CLI now uses frontend-equivalent JSON-RPC simulations, which were subsequently validated read-only on mainnet for the exact 1 HCHF request. Fresh hints can still become stale before consensus, so a later batch revert remains possible and safe by atomic rollback.

Step 5 — Build the two inners (approve first, contract call last)

Approve exactly the requested exact-or-revert redemption chunk from step 4d. Do not proceed with a truncated amount and do not approve a large multi-trove request. `batchify(client, key)` sets the batch key, signs with the payer key, and freezes (forcing node account 0.0.0). The browser-wallet implementation uses the distinct ephemeral-key sequence in §6.4.

CONFIRMED

```
const {
  AccountAllowanceApproveTransaction, ContractExecuteTransaction,
  TransactionId, AccountId
} = require("@hashgraph/sdk");

const TROVE_MANAGER = "0.0.6070118";
const HCHF_WRAPPER = AccountId.fromString("0.0.6070122"); // the spender

// inner 1 – native allowance tx (NOT a contract call).
// `amount` = exact requested chunk; step 4d required no truncation.
const approveTx = new AccountAllowanceApproveTransaction()
  .approveTokenAllowance(HCHF, client.operatorAccountId, HCHF_WRAPPER, amount)
  .setTransactionId(TransactionId.generate(client.operatorAccountId));
await approveTx.batchify(client, operatorKey);

// maxFeePercentage – 8-DECIMAL fixed point on THIS deployment (BaseMath
DECIMAL_PRECISION = 1e8):
// 100% = 1e8; the 0.5% borrow-fee floor = 500_000. Do not port percent constants from
// Ethereum-forked codebases – the decimal scale here is different.
const maxFeePercentage = 100_000_000; // = 100% – set your cap deliberately

// inner 2 – the SINGLE contract op; must be LAST. Gas 3,000,000 is a TEST DEFAULT –
// validate by local construction and the smallest practical mainnet pilot.
```

```

const redeemTx = new ContractExecuteTransaction()
  .setContractId(TROVE_MANAGER)
  .setGas(3_000_000)
  .setFunction("redeemCollateral",
    new ContractFunctionParameters()
      .addUint256(amount) // requested amount; step 4d proved it is not
truncated
      .addAddress(firstHint)
      .addAddress(partialUpperHint) // computed in 4d; zero only when partialNICR is
zero
      .addAddress(partialLowerHint) // computed in 4d; zero only when partialNICR is
zero
      .addUint256(partialNICR)
      .addUint256(1) // exact-or-revert safety shape
      .addUint256(maxFeePercentage)) // 8-dec percent: 100% = 1e8, 0.5% floor = 500_000
  .setTransactionId(TransactionId.generate(client.operatorAccountId));
await redeemTx.batchify(client, operatorKey);

```

Step 6 — Assemble, sign, execute

`addInnerTransaction` throws unless the inner is frozen with a `batchKey` set (that is what `batchify` did). The outer must then be signed by every distinct `batchKey` key plus the payer — with one key as `batchKey` on all inners, that key signs everything. **CONFIRMED**

```

const { BatchTransaction } = require("@hashgraph/sdk");

const batch = new BatchTransaction()
  .addInnerTransaction(approveTx) // first
  .addInnerTransaction(redeemTx); // last — the one contract op
batch.setTransactionId(TransactionId.generate(client.operatorAccountId));
await batch.freezeWith(client);
await batch.sign(operatorKey); // all batchKey keys + payer

const response = await batch.execute(client);
const receipt = await response.getReceipt(client);
console.log("batch status:", receipt.status.toString());
console.log("batch id:", batch.transactionId.toString());

```

Step 7 — Receipts and mirror-node verification

The JS SDK exposes **property getters** (`batch.innerTransactions` , `batch.innerTransactionIds`). Preserve both inner IDs, convert SDK form `0.0.x@seconds.nanos` to Mirror REST form `0.0.x-seconds-nanos` , and verify the parent plus each inner independently. Mirror transaction rows use `name` (not `type`). Confirm parent `ATOMICBATCH` , both inner statuses, exact token debit, and zero allowance before showing success. Any residue or debit mismatch is a critical incident requiring immediate revocation and investigation. **CONFIRMED**

```

const { TransactionReceiptQuery } = require("@hashgraph/sdk");
for (const id of batch.innerTransactionIds) { // property getters, not methods
  const r = await new TransactionReceiptQuery()
    .setTransactionId(id).execute(client);
}

```

```
console.log(id.toString(), "->", r.status.toString());
}
```

base: <https://mainnet-public.mirrornode.hedera.com/api/v1> # parent (ATOMICBATCH) + all inners, linked by parent_consensus_timestamp / nonce GET /transactions/{batchId} # residue check: the approved amount must be fully consumed (allowance value 0) GET /accounts/{owner}/allowances/tokens

Step 8 — The generic pattern for all 7 batched flows

Identical skeleton for every allowance-consuming flow — only the token, spender, target contract, function, and gas change:

- inner 1 (native):** exact amount and correct spender. For **closeTrove**, first read current entire debt including pending rewards and approve debt – 20 HCHF; never trust a user-entered/displayed debt value.
- inner 2 (the ONE contract op, last):** the flow's `ContractExecuteTransaction`.
- Assemble → freeze → sign (batchKey keys + payer) → execute → per-inner receipts → mirror-node residue check **immediately after commit**.

FLOW	BATCH CONTENTS	GAS — TEST DEFAULTS (RE-VALIDATE BEFORE ROLLOUT)
redeemCollateral the incident flow	[approve HCHF→wrapper exact] + [redeemCollateral] — one iteration, no truncation, chunk ≤1,780 HCHF; repeat for larger exits	3,000,000
provideToSP	[approve HCHF→wrapper exact] + [provideToSP]	800,000
repayHCHF	[approve exact] + [repayHCHF]	1,500,000
closeTrove	[freshly read entire debt; approve exact net debt = debt – 20 HCHF] + [closeTrove]	1,500,000
adjustTrove (repay or withdraw coll)	[approve exact] + [adjustTrove] (+ payable when adding collateral)	1,500,000
stake HLQT	[approve HLQT→HLQT wrapper exact] + [stake]	500,000
stake LP	[approve LP→SaucerSwapPool exact] + [stake]	500,000
No batch (no allowance): openTrove / addColl / withdrawColl / withdrawHCHF (single ContractExecute; set payable amount in HBAR for openTrove/addColl) · unstake / withdraw / claims · liquidations · claimCollateral. CONFIRMED		

closeTrove safety note. Read `TroveManager.getEntireDebtAndColl(user)` immediately before building; it includes pending redistribution rewards. Approve that debt minus 20 HCHF. A user-entered overestimate leaves sweepable residue. **CONFIRMED** (repo-verified)

Test defaults, not guarantees. The gas figures above are **TEST DEFAULTS** — validate locally and with the smallest practical mainnet pilot. Use live hints from HintHelpers and keep redemption `maxIterations = 1` for the exact-or-revert safety shape.

Step 9 — Troubleshooting

SYMPTOM	MEANING	FIX
<code>INNER_TRANSACTION_FAILED</code>	First non-SUCCESS inner aborted the whole batch; inners before it are still charged their fees.	Read per-inner receipts / mirror node to find which inner and why. For redeem: re-run the \$05 step-4 pre-flight (balance, hints, price freshness). Then rebuild — do not blindly retry.
<code>INVALID_TRANSACTION_BODY</code> (pre-check)	Constraint violated: more than one contract op, or the contract op is not the last inner.	Exactly one <code>ContractExecuteTransaction</code> , positioned last; the approve must be the native <code>AccountAllowanceApproveTransaction</code> .
<code>BUSY</code>	Node busy at submission.	Wait, rebuild with fresh inner transactionIds, resubmit. (Inners dedup individually.)
<code>PAYER_SIGNATURE_CHECK_FAILED</code>	Outer batch not signed by all distinct batchKey private keys + the payer.	With one batchKey on all inners: sign the outer with that same key (plus the payer signature).
SDK throws at <code>addInnerTransaction</code>	Inner not frozen or batchKey missing.	Call <code>batchify(client, key)</code> (or manual <code>setBatchKey</code> + <code>freezeWith</code> + <code>sign</code>) before adding.
Receipts query returns nothing for an inner id	Wrong id form or batch not yet processed.	Use <code>batch.innerTransactionIds</code> verbatim; poll with backoff; mirror node <code>GET /transactions/{batchId}</code> as ground truth.

06 Frontend recommendation — React + HashPack

6.1 Stack facts

DO NOT build on hashconnect — it is dead. Final release 3.0.14 (2025-10-02); the npm package carries the banner “shut down by 2026, upgrade to WalletConnect”; the GitHub repo is now 404. HashPack 14.3.0 (2026-06-24) removed the legacy HashConnect v2 protocol. Any hashconnect dependency is a liability and must be removed, not merely avoided. **CONFIRMED**

- **HashPack ≥ 14.4.0 (2026-07-17) supports batch transactions over WalletConnect** — “Dapps can now bundle an associate + transfer in a single approval.” 14.4.1 added dapp-auto-approval and removed the flaky WalletConnect auto-associate that “didn’t work well with batch transactions.” This is the minimum wallet version for the batched UX — and its batch signing is now **validated in practice** by the community mainnet test (§3.3). **CONFIRMED**
- **Connector: pin** `@hashgraph/hedera-wallet-connect` **1.5.1**. That exact resolved version produced the successful test. Its `DAppSigner.signTransaction` returns a new signed transaction; retaining that object is load-bearing. Relay uses `hedera_executeTransaction`. **CONFIRMED**
- **SDK: keep** `@hashgraph/sdk` **2.72.0**. Approval amounts are converted to `Long`; each batch inner is frozen with synthetic node `0.0.0`. **CONFIRMED**
- **What remains unverified:** every protocol flow other than the exact 1 HCHF redemption, other wallets/connectors, and wallet prompt rendering. The untested single-prompt path is removed, not exposed behind a flag. **UNVERIFIED**

6.2 Architecture: BatchTxBuilder around the existing builders

Build raw inners first, then centralize the proven signing sequence: generate one ephemeral ED25519 batch key; set its public key and an explicit payer transaction ID on each inner; call plain `freeze()`; call `DAppSigner.signTransaction(inner)` separately and retain every returned signed transaction; assemble those returned inners; `freezeWithSigner()` and wallet-sign the outer; retain the returned outer; add the ephemeral private-key signature locally; relay the fully signed bytes unchanged. Never use the wallet public key as batchKey and never call `freezeWithSigner()` on an inner. Reference TypeScript is under `code/frontend/`.

6.3 Flow → batch map (the frontend contract)

FLOW	BATCH CONTENTS	NOTES
openTrove	NO BATCH — single ContractExecute, <code>setPayableAmount(HBAR)</code>	no allowance involved
addColl / withdrawColl / withdrawHCHF	NO BATCH	no allowance
redeemCollateral	[Approve HCHF→wrapper 0.0.6070122 exact] + [ContractExecute redeemCollateral]	batch shape proven with exactly 1 HCHF ; one iteration, no truncation, each chunk ≤1,780 HCHF
provideToSP	[Approve HCHF→wrapper exact] + [provideToSP]	
repayHCHF	[Approve exact] + [repayHCHF]	
closeTrove	[Read current entire debt; approve exact debt – 20 HCHF] + [closeTrove]	Source-reviewed, not live-tested; never accept typed/displayed debt
adjustTrove (repay or withdraw coll)	[Approve exact] + [adjustTrove] (+ payable if adding coll)	allowance only on the repay side
stake HLQT	[Approve HLQT→wrapper 0.0.6070127 exact] + [stake]	
stake LP	[Approve LP→SaucerSwapPool 0.0.6070133 exact] + [stake]	
unstake / withdraw / claims	NO BATCH	no allowance; standalone calls keep working after March 2027
first-time token use	associate before entering the browser allowance flow	the tested browser builder intentionally accepts exactly two inners; the CLI-only three-inner variant remains unverified

6.4 Signing model — validated pattern, plus the #694 version caveat

Validated on mainnet (§3.3): exact ephemeral-key sequence, three prompts. Each inner is signed separately and replaced with the object returned by `DAppSigner.signTransaction`. The outer is then wallet-signed, replaced with the returned object, signed locally by the ephemeral batch key, and relayed unchanged. **CONFIRMED** (connector 1.5.1)

DAppSigner rebuild behavior is why returned objects are retained. Connector 1.5.1 serializes a transaction body, asks HashPack to sign it, and returns a newly decoded transaction containing that body and signature. Transaction-ID equality alone cannot prove byte equality and is not a valid issue-#694 guard. Pin the tested version and regression-test the full returned-object sequence on every upgrade.

The one-prompt `hedera_signAndExecuteTransaction` path is intentionally not present in the reference implementation. It has not been validated for this contract-call batch and should not be offered to users until separately proven.

6.5 UX requirements

- **Anticipate three signing prompts — and explain them before the wallet opens.** The validated wallet flow (§6.4) presents one prompt per inner transaction plus one for the outer batch (inner 1, inner 2, outer). A user expecting a single prompt will suspect a bug or an attack: the UI must say three prompts are coming and show a plain-language line for each inner (approve X HCHF to wrapper 0.0.6070122; redeem X HCHF for collateral). What HashPack renders *inside* each prompt is **UNVERIFIED** — until scenario T2 lands, treat wallet rendering as unknown and make the dapp-side preview authoritative.
- **Exact-amount display, always.** For closeTrove, show freshly read entire debt minus 20 HCHF. For redemption, allow only one-iteration, non-truncated chunks $\leq 1,780$ HCHF. Never round or accept typed debt.
- **Residue allowance check + emergency revoke.** Post-execution, query `/accounts/{user}/allowances/tokens` ; residue must be 0 and the exact token debit must match. If any residue is found, surface a one-click revoke and block further operations until cleared. This check is mandatory and cannot be omitted by a caller.
- **HashPack version gate.** Detect $< 14.4.0$ before connecting and show an upgrade path; do not silently degrade.
- **No non-atomic fallback.** Unknown versions and other wallets fail closed. A warning does not make an exposed standalone approval safe.
- **Honest fee estimate.** Show max fee covering the batch fee + each inner's full fee, and state that inner fees are charged even if the batch rolls back (§4.2).

6.6 Security checklist

- Exact-amount approvals only; no unlimited approvals anywhere in the fork (including legacy non-batched paths).
- **closeTrove approval equals freshly read entire debt minus 20 HCHF**; a user-entered overestimate is unsafe.
- batchKey is a fresh ephemeral ED25519 key generated in memory. It cannot authorize the account or inners; it signs the wallet-signed outer batch.

- All contract/token ids from the 2026-10-05 fingerprinted set (§05 step 3) — never from repo configs; ship a verified address manifest with the frontend.
- Canonical HLQT HTS 0.0.6070128 only — counterfeit-HLQT confusion is an active risk (§08); verify wrapper 0.0.6070127 is the spender before building.
- Enforce the invariant in code: exactly one contract op, last position; unit-test the builder against `INVALID_TRANSACTION_BODY` shapes.
- Redemption reuses the existing populated-transaction path, requires one iteration, no truncation, and a chunk $\leq 1,780$ HCHF.
- Log batch id + inner ids for every operation (audit + incident replay); mirror-node residue check runs before showing success.
- Pinned dependency versions; WalletConnect project id locked to the deployer's; supply-chain review when bumping the connector to 2.1.x.
- The bundled registry is mainnet-only. Validate changes with the smallest practical mainnet amount and compare exact mirror-node state.

6.7 HashPack test plan (6 scenarios)

#	SCENARIO	PROCEDURE	PASS CRITERIA
T1	Version gate	Connect with HashPack 14.3.x, then with $\geq 14.4.0$	14.3.x: blocked with explicit upgrade message, no silent fallback. $\geq 14.4.0$: proceeds.
T2	Signer-returned objects	Instrument connector 1.5.1 while signing both inners and the outer.	Exactly three signer calls; outer contains the two returned signed inners; final outer also has the ephemeral batch-key signature.
T3	Smallest mainnet happy path	Repeat the validated redemption with one iteration, no truncation, and the smallest practical amount.	Both inners SUCCESS; exact token debit; HBAR returned; allowance before/after 0/0.
T4	Atomic rollback	Batch a redeem that must revert (amount > balance)	Whole batch rolls back, <code>INNER_TRANSACTION_FAILED</code> , residue allowance = 0, both inner fees charged — and the fee estimate shown pre-tx covered reality.
T5	User rejection	Reject at the first of the three wallet prompts	Nothing submits; no allowance exists afterward; clean error state, retry offered.
T6	Unsupported wallet/version	Use an unknown version or non-HashPack wallet.	Flow is blocked; no standalone approval fallback is available.

07 Proposed community timeline — 20 weeks to the sunset

Week 1 begins 2026-10-05. This proposed community timeline treats “March 2027, estimated” as a hard stop, holds a user-exit completion buffer of **2027-01-31** (end of week 17), and keeps February 2027 free of any new dependency on contract-call inners. If the sunset slips, the buffer absorbs it; if it arrives early, only the router protects redemptions — and trove operations must already be finished, because a router cannot perform them for existing users (§3.2). Nothing here is mandated — contributors adopt dates by picking up the work. **INFERENCE** (date handling) / **RECOMMENDATION** (plan)

Track (week 1 = 2026-10-05)

W1W3W5W7W9W11W13W15W17W19W21 2027-01-31 · batched exits complete

Batch exit track — offline checks + smallest-amount mainnet validation

Frontend build — BatchTxBuilder + 7 flows

Community beta — controlled small-amount mainnet pilot

Mainnet rollout + user comms (incl. revoke campaign)

Buffer — mop-up, exceptions, non-HashPack users

Router track (parallel) — redemption-only router: build → audit → deploy

V2 decision & design — community migration analysis (parallel)

Milestones

≈ Mar 2027 (est.) · contract-call inners removed — HARD STOP

batch-exit track router track (redemption only) v2 track (dashed) 2027-01-31 buffer milestone
hard stop (estimated)

Dependency rules that make this plan safe: (1) weeks 1–8 complete offline construction checks and controlled smallest-amount mainnet scenarios T1–T6 against the verified registry; (2) by **2027-01-31** every user who needs a batched exit has completed it — including every closeTrove at the corrected net-debt approval amount (§2.1); (3) from February 2027 nothing new depends on contract-call inners — the buffer exists precisely because “March 2027” is an estimate, not a committed date; (4) the redemption-only router goes live inside the batch window (weeks 15–16) so redemptions cross from bridge to durable path before the stop; trove operations cannot be routed (§3.2) and must complete inside the batch window or await migration/v2; (5) user comms run continuously through rollout — allowance-revocation guidance stays live throughout (member-facing summary in §3.4).

08 Risks & open questions

RISK / OPEN QUESTION	DETAIL	MITIGATION / WHO PICKS IT UP
Sunset slips — or arrives early	"March 2027, estimated" is not a committed date. Earlier removal would cut the bridge short with users still exposed; a later one changes nothing about the router's value. INFERENCE	2027-01-31 completion buffer; router live by weeks 15–16; watch Hedera comms weekly (frontend contributors).
HashPack inner-rendering UX partially resolved	The community mainnet test observed three signing prompts (§3.3), but no public description exists of whether HashPack shows each inner's details or a summary inside those prompts. UNVERIFIED (rendering) / CONFIRMED (prompt count)	Scenario T2 in §6.7; dapp-side preview authoritative until resolved.
Wallet batch flow — validated, but only on one connector line	The assembled no-operator freeze + wallet-sign flow has no public example, but a community contributor reproduced it on mainnet (§3.3) with hedera-wallet-connect 1.5.1: three prompts, relay via <code>hedera_executeTransaction</code>. The same pattern on other connector versions is not yet evidenced. CONFIRMED (community-validated, 1.5.1)	Keep §6.7 T2–T3 as the regression gate; re-run on every connector version bump before mainnet use.
Fee-on-rollback economics	Each inner pays full fees even when the batch rolls back; a failed redeem costs real fees and leaves the user unredeemed. CONFIRMED	Read-only pre-flight (price/hints/balance) before building; honest max-fee UI; gas validated with controlled smallest-amount mainnet pilots.
Redemption preflight can fail or become stale	SDK <code>ContractCallQuery</code> cannot simulate <code>fetchPrice()</code> on this deployment because the path may write state; a live run stopped with <code>LOCAL_CALL_MODIFICATION_EXCEPTION</code>. Even with JSON-RPC <code>eth_call</code>, a price or sorted-list change before consensus can make a correct batch revert. The failed mainnet batch recorded in §05 confirms rollback. CONFIRMED (query failure and rollback) / INFERENCE (specific stale-state cause)	Use frontend-equivalent JSON-RPC <code>eth_call</code> for <code>fetchPrice</code> and all redemption hints, derive them immediately before signing, keep <code>maxIterations = 1</code> and exact/no-truncation checks, and explain that retries cost fees but leave no allowance.

RISK / OPEN QUESTION	DETAIL	MITIGATION / WHO PICKS IT UP
Bot adapts / other-client exposure	Batching closes the window only for batched flows on the upgraded frontend. Other frontends and dapps granting wrapper allowances remain sweepable; the bot is still active (~30–60 s cycle — community monitoring). CONFIRMED (bot) / INFERENCE (adaptation)	Fail closed in this client, provide an emergency revoke action, and keep monitoring and flagging (community IOC list + mirror-node watcher rules).
Counterfeit HLQT	Token-confusion risk when users approve HLQT: only the canonical HTS 0.0.6070128 via wrapper 0.0.6070127 is legitimate. CONFIRMED (existence of counterfeits confirmed in the community analysis)	Address manifest + id pinning in the builder; UI shows verified ids.
Repo/mainnet config mismatch	Live mainnet deployment was never committed; committed <code>hedera.json</code> is testnet-era and mislabeled. Building from repo configs targets the wrong contracts. CONFIRMED	Fingerprinted set only (§05 step 3); publish the verified manifest; fix or clearly tombstone repo configs. RECOMMENDATION
Router scope (narrowed, community-2.0)	Durable router = permissionless redemption only : <code>closeTrove/adjustTrove</code> act on <code>msg.sender</code> , so a router would be the caller, not the trove owner — it cannot perform those operations for existing users. SP exits, unstaking, claims, and <code>claimCollateral</code> need no router (no allowance involved; standalone calls keep working after March 2027). Existing-user trove operations = batches until cutoff / migration / v2. RECOMMENDATION	Scope the router to redemption (§3.2); keep trove-exit completion inside the 2027-01-31 buffer (§07); migration/v2 track covers the rest.
Connector version	Fork pins <code>hedera-wallet-connect</code> ^1.4.2; the validated signing path ran on 1.5.1 (satisfies ^1.4.2); latest is 2.1.x. CONFIRMED	Decide upgrade vs. pin during weeks 3–6; either way: no hashconnect, and re-run §6.7 T2–T3 on the new version before mainnet.

RISK / OPEN QUESTION	DETAIL	MITIGATION / WHO PICKS IT UP
#694 as a version caveat	<code>DAppSigner.signTransaction</code> rebuilds the <code>ContractExecute</code> body on some connector versions, breaking naive sign-then-submit. The validated 1.5.1 per-transaction path is unaffected; the caveat binds on version bumps, not today. CONFIRMED (issue open) / INFERENCE (version scoping)	Re-test the §6.4 signing path at each connector bump; until a version fails, the validated per-transaction pattern stands.

09 Evidence & verification appendix

9.1 Primary sources (all API claims verified against these in this research round)

CLAIM AREA	SOURCE	PATH / URL
HIP-551 spec (Final; release v0.66.0; mainnet re-enabled 2025-10-09)	Hedera Improvement Proposal	https://hips.hedera.com/hip/hip-551
One-contract-op-last rule; forbidden inner types; fees-on-rollback; batchKey signing	Hedera documentation "Create a Batch Transaction" + consensus-node source (<code>AtomicBatchHandler.pureChecks</code> , hiero-ledger/hedera-consensus-node)	docs.hedera.com · github.com/hiero-ledger/hedera-consensus-node
JS SDK BatchTransaction API (<code>addInnerTransaction</code> , <code>batchify</code> , property getters; ≥2.64.0; also <code>@hiero-ledger/sdk</code> 2.89.x)	SDK JSDoc + repo examples	npm <code>@hashgraph/sdk</code> · SDK repo <code>examples/batch-tx.js</code> (batchify + multi-batchKey patterns)
Complete official batch example	Hedera tutorial repo	github.com/hedera-dev/tutorial-js-hip-551-batch-transaction (<code>mintFractionalAsset.js</code>)
March 2027 deprecation (quotes in §3.1)	Official Hedera blog, 2026-09-22 — "Atomic Batch Transactions No Longer Support Smart Contract Calls"	hedera.com/blog (2026-09-22)
2023 precedent (emergency shutoff, v0.34.5, HSCS Security Model v2)	Official Hedera post-mortem, 2023- 03-11	https://hedera.com/blog/analysis-remediation-of-the-precompile-attack-on-the-hedera-network/
hashconnect deprecated / shut down; repo 404; HashPack 14.3.0 removed legacy protocol	npm registry + GitHub + HashPack patch notes	npm <code>hashconnect</code> (final 3.0.14, 2025-10-02) · HashPack patch notes 14.3.0 (2026-06-24)

CLAIM AREA	SOURCE	PATH / URL
HashPack ≥14.4.0 batch support; 14.4.1 dapp-auto-approval; auto-associate removal	HashPack patch notes	HashPack 14.4.0 (2026-07-17), 14.4.1
HIP-820 connector + methods; #694 bug	@hashgraph/hedera-wallet-connect (2.1.x in 2026) + issue tracker	npm/github @hashgraph/hedera-wallet-connect · issue #694
blokk fork stack (hedera-wallet-connect ^1.4.2 + sdk 2.72.0)	Repo inspection	github.com/blokk-studio/HLiquidity · github.com/SwisscoastAG/HLiquidity
Reference implementation (CLI + frontend TypeScript)	Community reference code	published alongside this document in the repository under code/ — github.com/SwisscoastAG/HLiquidity
Mainnet contract set (fingerprinted 2026-10-05)	On-chain fingerprinting vs. both repos	ids as listed in §2/§5 of this document
Community mainnet validation of the batched redemption — reproduction facts, three signing prompts, zero residue (§3.3)	Community contributor mainnet test (contributor unnamed)	the GitHub issue in github.com/SwisscoastAG/HLiquidity
closeTrove net-debt burn and 8-decimal maxFeePercentage (community-2.0 corrections)	Repo-verified against the deployed contract source (BaseMath.sol DECIMAL_PRECISION = 1e8; BorrowerOperations.closeTrove GasPool compensation)	github.com/SwisscoastAG/HLiquidity · github.com/blokk-studio/HLiquidity
Incident campaign figures (victims, waves, \$ ledger, open allowances)	Community forensic analysis of mainnet data (methodology in §2/appendix)	mirror-node REST + HashScan, endpoints in §9.2

9.2 Measurement endpoints

base: <https://mainnet-public.mirrornode.hedera.com/api/v1> # batch verification: parent (type ATOMICBATCH) + inners (batch_key, parent_consensus_timestamp, nonce) GET /transactions/{batchId} # per-owner allowance state — residue must be 0 after a committed batch GET /accounts/{accountId}/allowances/tokens # reference incident tx (the 1,800 HCHF drain, 2026-09-29) GET /transactions/0.0.995584-1790680920-326001014 # same transaction, browser view <https://hashscan.io/mainnet/transaction/0.0.995584-1790680920-326001014>

9.3 Verification status of load-bearing claims

- **CONFIRMED** — HIP-551 semantics and limits; the one-contract-op-last rule; fee-on-rollback; batchKey signing model; the deprecation blog and its exact quotes; hashconnect deprecation; HashPack $\geq 14.4.0$ batch support; issue #694; blokk fork stack; mainnet contract set; all incident/\$ figures from community forensic analysis of public mainnet data. **Community-validated on mainnet (§3.3):** the assembled hedera-wallet-connect + BatchTransaction flow; the no-operator freeze + wallet-signing pattern; HashPack $\geq 14.4.0$ batch signing in practice via connector 1.5.1 (three signing prompts observed; signed bytes relayed via `hedera_executeTransaction`; exactly 100,000,000 units consumed; zero residue allowance).
- **RECOMMENDATION** — batch-every-flow policy; router design (permissionless-redemption scope — §3.2/§08); v2 endgame per the community migration analysis (summarized in §3.2); UX, security, and test-plan specifications; timeline structure.
- **INFERENCE** — race-elimination claim (no inter-transaction window exists by construction; non-batched and non-upgraded surfaces remain exposed); treating “March 2027, estimated” as a hard planning date; bot-adaptation outlook.
- **UNVERIFIED** — how HashPack renders the inner details inside each signing prompt; wallet/connector combinations other than the validated HashPack + hedera-wallet-connect 1.5.1 path.

9.4 Changelog

community-2.3 (2026-10-05). Live CLI preflight result incorporated:

- **JSON-RPC redemption simulation:** replaced the SDK `ContractCallQuery` redemption preflight with frontend-equivalent JSON-RPC `eth_call`. A mainnet run proved that SDK local execution rejects `fetchPrice()` with `LOCAL_CALL_MODIFICATION_EXCEPTION` because that path may update state. The rejected preflight created no batch and no allowance.
- **Read-only mainnet validation:** the replacement path returned execution price `8439828`, first hint `0x...216440`, partial NICR `3999701692937646830925`, upper hint `0x...1347B9`, lower hint `0x...216440`, and an untruncated amount of `100000000` raw units for the exact 1 HCHF request.

- **Raw key parsing:** DER keys use `fromStringDer` ; raw HashPack keys use explicit `fromStringED25519` or `fromStringECDSA` , eliminating the deprecated ambiguous parser warning.

community-2.2 (2026-10-05). CLI redemption preflight corrected from the failed mainnet rollback test:

- **Execution-equivalent price:** replaced `PriceFeed.lastGoodPrice()` with a read-only simulation of `fetchPrice()` , matching both `TroveManager.redeemCollateral` and the established frontend.
- **Partial insertion hints:** the CLI now performs `getApproxHint` plus `SortedTrove.findInsertPosition` and passes the returned upper/lower hints. Zero/zero is retained only when the partial NICR is zero.
- **Rollback evidence:** parent `0.0.4093645@1791227892.358580307` failed with `INNER_TRANSACTION_FAILED / TroveManager: Unable to redeem any amount` ; the approval was rolled back, the HCHF balance was unchanged, and allowance remained zero. This proves safety of rollback, not liveness of the old CLI recipe.
- **Tests:** focused tests cover `fetchPrice` , non-zero and zero partial-NICR paths, exact upper/lower propagation, and bounded approximation-query chunks.

community-2.1 (2026-10-05). Reference code and instructions aligned to the successful mainnet transaction. One line per correction:

- **Mainnet validation — PROVEN (§3.3):** a community contributor reproduced the batched redemption on mainnet — `[CryptoApproveAllowance 1 HCHF] + [TroveManager.redeemCollateral]` as the final inner under one parent `AtomicBatch`; the batch succeeded, exactly 100,000,000 units were consumed and burned, HBAR was returned, and the residue allowance was zero; the mirror record confirms ordering and shared parent. Evidence in the GitHub issue; the contributor remains unnamed unless they request attribution. Three signing prompts observed (inner 1, inner 2, outer batch).
- **closeTrove approval amount (safety-critical, §2.1/§05/§6.3/§6.6):** approve exactly net debt (debt – 20 HCHF); the GasPool provides the 20 HCHF gas compensation. Approving the full debt leaves a 20 HCHF stealable allowance.
- **maxFeePercentage decimals (§05):** corrected to the deployment's 8-decimal fixed point (`BaseMath.DECIMAL_PRECISION = 1e8`): 100% = 1e8, 0.5% floor = 500_000.
- **Redemption exactness (§05 steps 4d–5, §6.5):** permit only a non-truncated request no larger than 1,780 HCHF with `maxIterations = 1` ; otherwise abort before signing. Larger exits use repeated batches. This is the shape proven by the successful 1 HCHF mainnet test and prevents a successful call from consuming less than its allowance.
- **Router scope narrowed (§3.2/§07/§08):** durable router = permissionless redemption ONLY — `closeTrove/adjustTrove` act on `msg.sender` and cannot be routed for existing users; SP

exits, unstaking, and claims need no router (standalone calls keep working after March 2027); existing-user trove operations = batches until cutoff / migration / v2.

- **Signing patterns (§6.4/§6.5/§6.7):** generate a one-use ED25519 batch key; freeze and wallet-sign each inner separately, retaining the returned transaction objects; assemble and wallet-sign the outer; add the ephemeral signature locally; relay the fully signed bytes with `hedera_executeTransaction`. Three HashPack prompts via hedera-wallet-connect 1.5.1 were validated on mainnet. Alternate and non-atomic submission paths are removed.
- **Test defaults labeled (§05):** fixed gas values are test defaults; the bundled addresses are mainnet-only, so validate construction offline and then use controlled smallest-amount mainnet pilots.

community-1.0 (2026-10-05): initial public community edition. community-2.0 added the first correction round. community-2.1 aligned the wallet path to the successful mainnet transaction. community-2.2 added live redemption hints. community-2.3 is the current ground-truth edition; corrections always via GitHub issues.

Disclaimer. This is an independent, community-produced engineering analysis based solely on public Hedera mainnet data (mirror-node REST API, HashScan), official Hedera sources (HIP-551, documentation, consensus-node source, the 2026-09-22 deprecation blog, the 2023 post-mortem), npm registry and wallet patch notes, and the cited public GitHub repositories. No entity operates the HLIquity protocol; this document was produced by community contributors from public mainnet data. It is not affiliated with, or endorsed by, Swisscoast AG, HashPack, Hedera, or any audited protocol. Address references describe on-chain accounts; they are not accusations against identified natural persons or legal entities. Not financial advice. Corrections via GitHub issues only.